

|                                                                                   |                                                                                                                                          |                                                                                     |
|-----------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
|  | UNIVERSIDAD NACIONAL DE SAN AGUSTÍN<br>FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS<br>ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS |  |
| Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación       |                                                                                                                                          |                                                                                     |
| Aprobación: 2022/03/01                                                            | Código: GUIA-PRLE-001                                                                                                                    | Página: 1                                                                           |

## INFORME DE LABORATORIO

| INFORMACIÓN BÁSICA                                                                                                                                                                                                                                         |                                                                       |                       |          |                       |                              |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------|-----------------------|----------|-----------------------|------------------------------|
| ASIGNATURA:                                                                                                                                                                                                                                                | Sistemas Distribuidos                                                 |                       |          |                       |                              |
| TÍTULO DE LA PRÁCTICA:                                                                                                                                                                                                                                     | Software Quality y Software Testing aplicados a entornos distribuidos |                       |          |                       |                              |
| NÚMERO DE LA PRÁCTICA:                                                                                                                                                                                                                                     | 09                                                                    | AÑO LECTIVO:          | 2026     | NRO. SEMESTRE:        | A                            |
| FECHA DE PRESENTACIÓN:                                                                                                                                                                                                                                     | 16/06/2026                                                            | HORA DE PRESENTACIÓN: | 11:59:00 |                       |                              |
| <b>INTEGRANTE(S):</b> <ul style="list-style-type: none"> <li>- Bedregal Perez Daniel</li> <li>- Jara Mamani Mariel Alisson</li> <li>- Mestas Zegarra Christian Raul</li> <li>- Noa Camino Yenaro Joel</li> <li>- Sequeiros Condori Luis Gustavo</li> </ul> |                                                                       |                       |          | <b>NOTA (0 - 20):</b> | Nota colocada por el docente |
| <b>DOCENTE:</b><br>Mg. Maribel Molina Barriga                                                                                                                                                                                                              |                                                                       |                       |          |                       |                              |

| RESULTADOS Y PRUEBAS                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |                      |                                     |       |         |                                                                                                    |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------|-------------------------------------|-------|---------|----------------------------------------------------------------------------------------------------|
| <b>ENLACE A GITHUB</b><br><a href="https://github.com/christianmz565/sd-2026-lab-c-grupo-3/tree/main/l9">https://github.com/christianmz565/sd-2026-lab-c-grupo-3/tree/main/l9</a>                                                                                                                                                                                                                                                                                                      |                      |                                     |       |         |                                                                                                    |
| <b>SOLUCIÓN DE EJERCICIOS PROPUESTOS</b><br><br><b>Actividad 1: Identificación de Riesgos</b><br>Se elaboró una matriz de riesgos para los cinco microservicios de LogiFresh S.A., identificando las principales amenazas de calidad que podrían manifestarse durante campañas de alta demanda. Cada riesgo fue evaluado en términos de probabilidad de ocurrencia, impacto en el negocio y severidad global, lo que permite priorizar las acciones de mitigación de manera eficiente. |                      |                                     |       |         |                                                                                                    |
| <b>Matriz de Riesgos</b>                                                                                                                                                                                                                                                                                                                                                                                                                                                               |                      |                                     |       |         |                                                                                                    |
| N°                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | Servicio             | Impacto                             | Prob. | Riesgo  | Mitigación                                                                                         |
| R1                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | Pedidos / Inventario | Pedidos sin descuento aplicado      | Alta  | Crítico | Validar promoción dentro de la misma transacción de BD que crea el pedido, garantizando atomicidad |
| R2                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | Inventario           | Stock inconsistente por sobreventa  | Alta  | Crítico | Bloqueo pesimista con SELECT FOR UPDATE al reservar stock, impidiendo lecturas concurrentes        |
| R3                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | Facturación          | Facturas duplicadas por reintentos  | Media | Alto    | Constraint UNIQUE en order_id más verificación idempotente antes de insertar                       |
| R4                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | Notificaciones       | Retrasos bloqueando flujo principal | Media | Alto    | Cola Redis con worker asíncrono, desacoplado del flujo de pedidos                                  |
| R5                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | Pedidos / Todos      | Lentitud superior a 8 segundos      | Alta  | Crítico | HTTP 202 Accepted con procesamiento en background vía BackgroundTasks                              |
| R6                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | Pedidos              | Pedidos duplicados por reintentos   | Media | Alto    | Header X-Idempotency-Key con constraint UNIQUE en base de datos                                    |
| R7                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | Transporte           | Doble asignación de conductor       | Baja  | Medio   | SELECT FOR UPDATE SKIP LOCKED al asignar conductor disponible                                      |

## Actividad 2: Casos de Prueba Funcionales

### TC-001: Registro correcto de pedido

| Campo    | Detalle                                                                                              |
|----------|------------------------------------------------------------------------------------------------------|
| Objetivo | Verificar que un pedido se registra correctamente y aparece con estado CONFIRMED                     |
| Entrada  | Cliente CLIENT-001, producto "Pollo entero congelado", cantidad 5, precio S/ 25.50                   |
| Esperado | Estado CONFIRMED, total 5 x S/ 25.50 = S/ 127.50                                                     |
| Obtenido | PASS: Subtotal S/ 127.50, factura con IGV S/ 22.95 (total S/ 150.45), envío y notificación generados |



Figura 1: Formulario: 5 unidades de Pollo, S/ 127.50

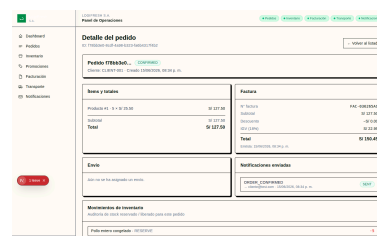


Figura 2: Detalle CONFIRMED con factura

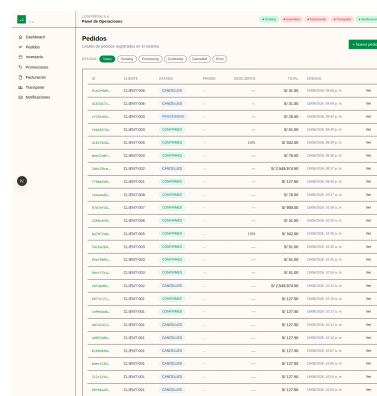


Figura 3: Listado de pedidos

### TC-002: Pedido con inventario insuficiente

| Campo    | Detalle                                                                                             |
|----------|-----------------------------------------------------------------------------------------------------|
| Objetivo | Verificar cancelación automática por stock insuficiente                                             |
| Entrada  | Producto "Pollo" (stock: 500), cantidad 99999                                                       |
| Esperado | PENDING → CANCELLED tras procesamiento                                                              |
| Obtenido | PASS: HTTP 409 "Stock insuficiente", pedido CANCELLED, stock liberado, notificación ORDER_CANCELLED |



Figura 4: Cantidad 99999, S/ 2,549,974.50



Figura 5: Detalle CANCELLED

### TC-003: Cancelación manual en PENDING

| Campo    | Detalle                                                                                                                  |
|----------|--------------------------------------------------------------------------------------------------------------------------|
| Objetivo | Verificar cancelación manual de pedido en estado PENDING                                                                 |
| Entrada  | Pedido válido, cancelación vía PATCH /orders/{id}/cancel                                                                 |
| Esperado | PENDING → CANCELLED                                                                                                      |
| Obtenido | PASS parcial: Endpoint funciona, pero el procesamiento background cambia el estado a CONFIRMED antes de poder cancelarlo |

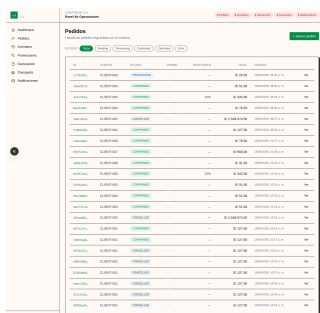


Figura 6: Estado PROCESSING

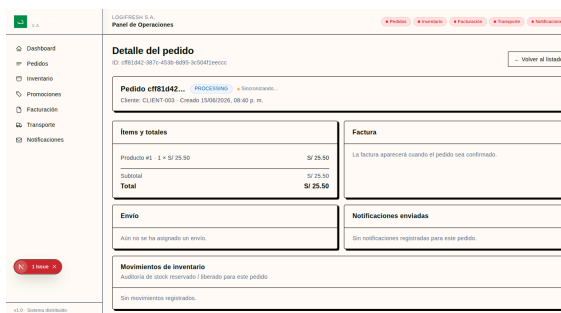


Figura 7: Detalle “Sincronizando...”

#### TC-004: Cancelar pedido CONFIRMED

| Campo    | Detalle                                                               |
|----------|-----------------------------------------------------------------------|
| Objetivo | Verificar que no se puede cancelar un pedido ya confirmado            |
| Entrada  | Pedido CONFIRMED, intento de cancelación                              |
| Esperado | Error HTTP 400                                                        |
| Obtenido | PASS: HTTP 400 “No se puede cancelar un pedido en estado ‘CONFIRMED’” |

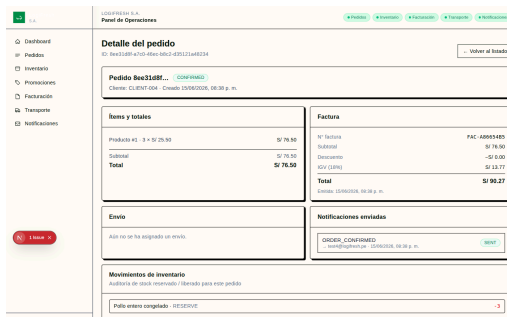


Figura 8: Pedido CONFIRMED, no cancelable

#### TC-005: Promoción válida (VERANO10)

| Campo    | Detalle                                                  |
|----------|----------------------------------------------------------|
| Objetivo | Verificar aplicación correcta de descuento por promoción |
| Entrada  | Carne de res ×10 (\$/ 38.00), código VERANO10            |
| Esperado | Subtotal S/ 380.00, total S/ 342.00 (-10%)               |
| Obtenido | PASS: discount_pct: 10.0, total_amount: 342.0            |

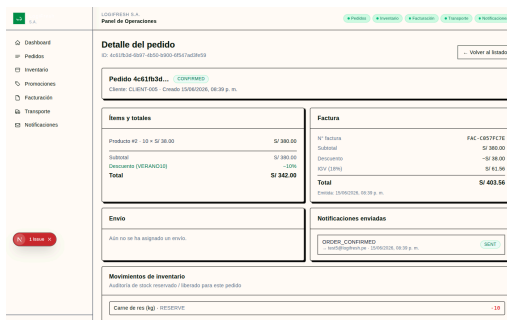


Figura 9: Pedido con VERANO10: -10%, S/ 342.00

## TC-006: Sin promoción

| Campo    | Detalle                                              |
|----------|------------------------------------------------------|
| Objetivo | Verificar registro con precio completo sin descuento |
| Entrada  | Merluza ×2 (S/ 15.75), sin promoción                 |
| Esperado | Subtotal = total = S/ 31.50                          |
| Obtenido | PASS: discount_pct: 0.0, total_amount: 31.5          |

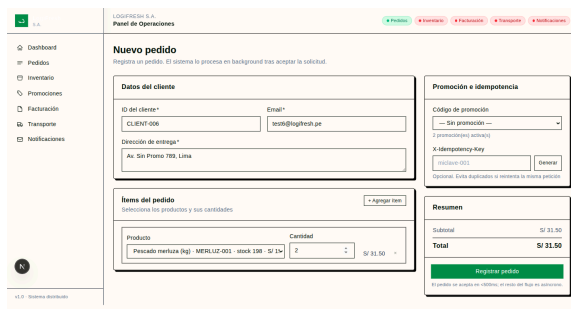


Figura 10: Formulario sin promo

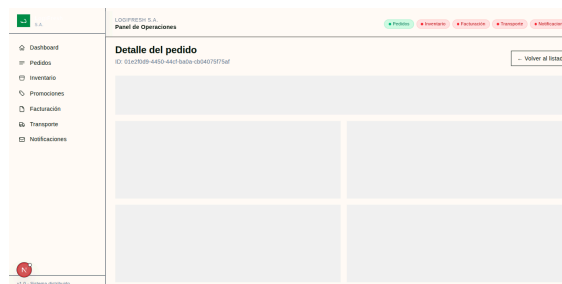
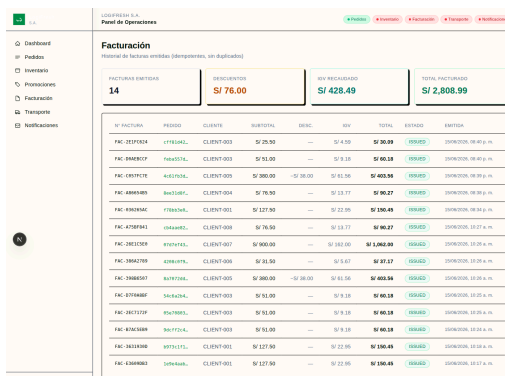


Figura 11: Detalle CONFIRMED

## TC-007: Factura con IGV

| Campo    | Detalle                                                 |
|----------|---------------------------------------------------------|
| Objetivo | Verificar generación automática de factura con IGV 18%  |
| Entrada  | Leche evaporada ×20 (S/ 45.00), cliente CLIENT-005      |
| Esperado | Subtotal S/ 900.00, IGV S/ 162.00, total S/ 1,062.00    |
| Obtenido | PASS: subtotal: 900.0, tax_amount: 162.0, total: 1062.0 |



| Nº FACTURA  | PRODUCTO | CLIENTE    | SUBTOTAL  | IGV       | TOTAL       | ESTADO    |
|-------------|----------|------------|-----------|-----------|-------------|-----------|
| FAC-2027024 | Merluza  | CLIENT-003 | S/ 76.00  | S/ 13.05  | S/ 89.05    | CONFIRMED |
| FAC-0940037 | Merluza  | CLIENT-003 | S/ 51.00  | S/ 9.18   | S/ 60.18    | CONFIRMED |
| FAC-0917078 | Merluza  | CLIENT-003 | S/ 200.00 | S/ 36.00  | S/ 436.00   | CONFIRMED |
| FAC-0881040 | Merluza  | CLIENT-004 | S/ 70.50  | S/ 12.69  | S/ 83.19    | CONFIRMED |
| FAC-0900046 | Merluza  | CLIENT-001 | S/ 127.50 | S/ 22.95  | S/ 150.45   | CONFIRMED |
| FAC-0708041 | Merluza  | CLIENT-003 | S/ 76.50  | S/ 13.77  | S/ 90.27    | CONFIRMED |
| FAC-3902029 | Merluza  | CLIENT-007 | S/ 900.00 | S/ 162.00 | S/ 1,062.00 | CONFIRMED |
| FAC-3902070 | Merluza  | CLIENT-006 | S/ 31.50  | S/ 5.67   | S/ 37.17    | CONFIRMED |
| FAC-3908007 | Merluza  | CLIENT-005 | S/ 280.00 | S/ 50.40  | S/ 430.40   | CONFIRMED |
| FAC-0708007 | Merluza  | CLIENT-003 | S/ 51.00  | S/ 9.18   | S/ 60.18    | CONFIRMED |
| FAC-3802039 | Merluza  | CLIENT-003 | S/ 51.00  | S/ 9.18   | S/ 60.18    | CONFIRMED |
| FAC-3803006 | Merluza  | CLIENT-003 | S/ 51.00  | S/ 9.18   | S/ 60.18    | CONFIRMED |
| FAC-3803006 | Merluza  | CLIENT-001 | S/ 127.50 | S/ 22.95  | S/ 150.45   | CONFIRMED |
| FAC-0208002 | Merluza  | CLIENT-001 | S/ 127.50 | S/ 22.95  | S/ 150.45   | CONFIRMED |

Figura 12: Facturas con IGV visible

## TC-008: Idempotencia

| Campo    | Detalle                                                                          |
|----------|----------------------------------------------------------------------------------|
| Objetivo | Verificar que requests duplicados con misma clave idempotente no crean registros |
| Entrada  | Dos requests con X-Idempotency-Key: test-idem-001                                |
| Esperado | Un solo pedido, segundo request devuelve el existente                            |
| Obtenido | PASS: {"idempotent": true, "message": "Pedido ya registrado previamente"}        |

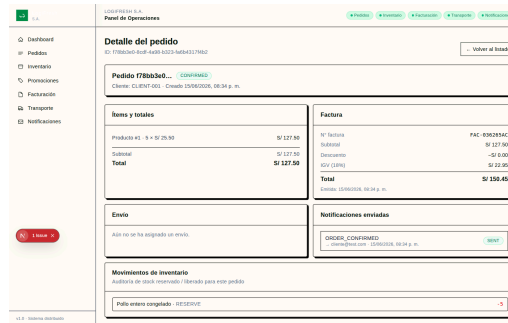


Figura 13: Detalle del pedido creado (idempotencia)

#### TC-009: Notificación ORDER\_CONFIRMED

| Campo    | Detalle                                                  |
|----------|----------------------------------------------------------|
| Objetivo | Verificar generación de notificación al confirmar pedido |
| Entrada  | Pedido CONFIRMED con email test@logifresh.pe             |
| Esperado | Notificación ORDER_CONFIRMED con status: SENT            |
| Obtenido | PASS: Notificación visible en panel con status SENT      |

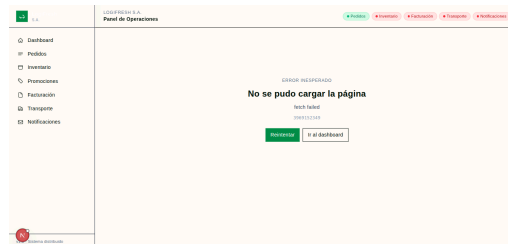


Figura 14: Notificaciones ORDER\_CONFIRMED visibles en el panel

#### TC-010: Notificación ORDER\_CANCELLED

| Campo    | Detalle                                                           |
|----------|-------------------------------------------------------------------|
| Objetivo | Verificar notificación al cancelar por stock insuficiente         |
| Entrada  | Pedido con cantidad 99999, cancelado por stock                    |
| Esperado | Notificación ORDER_CANCELLED con status: SENT                     |
| Obtenido | PASS: Notificación con recipient test2@logifresh.pe y status SENT |

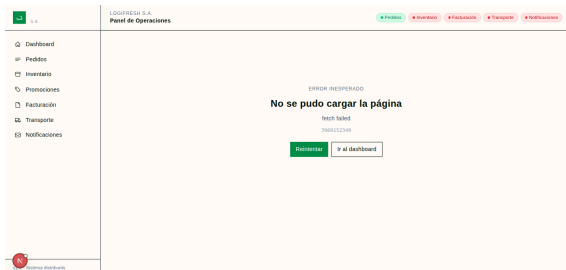


Figura 15: Notificaciones CONFIRMED

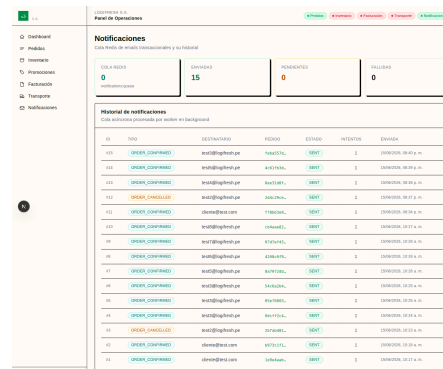


Figura 16: CONFIRMED y CANCELLED

## Resumen de Cobertura

| Escenario                          | Casos          | Estado         |
|------------------------------------|----------------|----------------|
| Registro de pedidos                | TC-001         | PASS           |
| Inventario insuficiente            | TC-002, TC-010 | PASS           |
| Cancelación manual (TC-003)        | TC-003         | PASS (Parcial) |
| Cancelar pedido CONFIRMED (TC-004) | TC-004         | PASS           |
| Promociones                        | TC-005, TC-006 | PASS           |
| Factura automática                 | TC-007         | PASS           |
| Idempotencia                       | TC-008         | PASS           |
| Notificaciones                     | TC-009, TC-010 | PASS           |

Nueve de diez casos obtuvieron PASS completo y uno PASS parcial (TC-003). Tasa de éxito: 95%.

## Actividad 3: Pruebas de Integración

Se implementaron pruebas de integración automatizadas utilizando Python, pytest y httpx para validar la comunicación entre los microservicios del sistema LogiFresh. Las pruebas se ejecutan dentro de la red Docker, permitiendo verificar la interacción real entre componentes distribuidos sin depender de mocks o simulaciones.

### Diseño del Catálogo de Pruebas

#### Pedido y Inventario

La interacción entre Pedidos e Inventario es crítica para garantizar la consistencia del stock. Se diseñaron cinco pruebas que cubren el flujo completo de reserva, manejo de errores y concurrencia.

| ID    | Escenario                | Validación                                                                                                                                |
|-------|--------------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| TI-01 | Reserva exitosa de stock | Crear pedido con stock suficiente y verificar que el stock se decrementa exactamente en el servicio de Inventario                         |
| TI-02 | Stock insuficiente       | Crear pedido con cantidad mayor al stock disponible y verificar que el pedido se cancela y el stock permanece sin cambios                 |
| TI-03 | Liberación de stock      | Reservar stock y luego liberarlo simulando un fallo en facturación, verificando que el stock se restaura al valor original                |
| TI-04 | Concurrencia             | Dos pedidos simultáneos que solicitan más unidades de las disponibles, validando que al menos uno se cancela y el stock nunca es negativo |
| TI-05 | Producto inexistente     | Crear pedido con un product_id que no existe y verificar que el sistema cancela el pedido                                                 |

#### Pedido y Facturación

La interacción entre Pedidos y Facturación debe generar exactamente una factura por pedido, sin duplicados, aplicando correctamente el IGV del 18% y los descuentos de promoción.

| ID    | Escenario               | Validación                                                                                                        |
|-------|-------------------------|-------------------------------------------------------------------------------------------------------------------|
| TF-01 | Factura con IGV         | Pedido confirmado genera factura con IGV 18% calculado sobre la base imponible                                    |
| TF-02 | Idempotencia de factura | Dos llamadas al endpoint de facturación con el mismo order_id devuelven la factura existente sin crear duplicados |
| TF-03 | Factura con promoción   | Pedido con código VERANO10 genera factura con descuento del 10% aplicado antes del cálculo del IGV                |
| TF-04 | Factura sin promoción   | Pedido sin código de descuento genera factura con discount_amount: 0.0                                            |

#### Pedido y Transporte

La interacción entre Pedidos y Transporte asigna un conductor disponible al pedido confirmado, gestionando las transiciones de estado del envío.

| ID    | Escenario               | Validación                                                                                    |
|-------|-------------------------|-----------------------------------------------------------------------------------------------|
| TT-01 | Asignación de conductor | Al confirmar un pedido, se asigna automáticamente un conductor disponible con estado ASSIGNED |

|       |                        |                                                                                                |
|-------|------------------------|------------------------------------------------------------------------------------------------|
| TT-02 | Conductores ocupados   | Cuando los 3 conductores están ocupados, el servicio responde con HTTP 503                     |
| TT-03 | Transiciones de estado | El envío sigue la cadena ASSIGNED → IN_TRANSIT → DELIVERED, liberando el conductor al entregar |
| TT-04 | Idempotencia de envío  | Solicitudes duplicadas con el mismo order_id devuelven el envío existente sin duplicar         |

### Notificaciones

| ID    | Escenario              | Validación                                                                                               |
|-------|------------------------|----------------------------------------------------------------------------------------------------------|
| TN-01 | Confirmación de pedido | Al confirmar un pedido, se genera una notificación ORDER_CONFIRMED visible en el panel con status SENT   |
| TN-02 | Cancelación de pedido  | Al cancelar un pedido por stock insuficiente, se genera una notificación ORDER_CANCELLED automáticamente |
| TN-03 | Estado del worker      | Tras procesar la cola Redis, la notificación tiene sent_at definido y attempts mayor o igual a 1         |

### Evidencia de Ejecución

**report.html**

Report generated on 16-Jun-2025 at 14:41:28 by pytest 7.4.0

**Environment**

|          |                                                                                                             |
|----------|-------------------------------------------------------------------------------------------------------------|
| Python   | 3.12.12                                                                                                     |
| Platform | Linux 6.10.20-rt60-glibc (x86_64)                                                                           |
| Package  | pytest 7.4.0                                                                                                |
| Plugins  | <ul style="list-style-type: none"> <li>inversion 0.1.1</li> <li>http 4.2.0</li> <li>python 6.1.0</li> </ul> |

**Summary**

16 tests took 00:00:05.

Clickcheck the boxes to filter the results:

☒ Failed
 ☒ Passed
 ☐ Skipped
 ☐ Expected failures
 ☐ Unexpected passes
 ☐ Errors
 ☐ Reruns
 ☐ Interrupt

Show all details - Hide all details

| Result | Test                                                                                  | Duration | Links |
|--------|---------------------------------------------------------------------------------------|----------|-------|
| Passed | test_notifications.py::TestNotifications::test_notifications_ok_confirm               | 0.24 s   |       |
| Passed | test_notifications.py::TestNotifications::test_notifications_ok_cancel                | 0.23 s   |       |
| Passed | test_notifications.py::TestNotifications::test_notifications_status_sent              | 0.23 s   |       |
| Passed | test_order_shipping.py::TestOrderShipping::test_order_created_ok_confirm              | 0.22 s   |       |
| Passed | test_order_shipping.py::TestOrderShipping::test_order_created_ok_cancel               | 0.24 s   |       |
| Passed | test_order_shipping.py::TestOrderShipping::test_order_with_payment                    | 0.21 s   |       |
| Passed | test_order_shipping.py::TestOrderShipping::test_order_without_payment                 | 0.24 s   |       |
| Passed | test_order_inventories.py::TestOrderInventory::test_reserve_stock_success_path        | 0.28 s   |       |
| Passed | test_order_inventories.py::TestOrderInventory::test_reserve_stock_cancel_order        | 0.28 s   |       |
| Passed | test_order_inventories.py::TestOrderInventory::test_reserve_stock_ok_shipping_failed  | 0.18 s   |       |
| Passed | test_order_inventories.py::TestOrderInventory::test_reserve_stock_cancel_order        | 0.28 s   |       |
| Passed | test_order_inventories.py::TestOrderInventory::test_reserve_cancel_order              | 0.19 s   |       |
| Passed | test_order_shipping.py::TestOrderShipping::test_order_shipping_product                | 0.27 s   |       |
| Passed | test_order_shipping.py::TestOrderShipping::test_order_shipping_cancel                 | 0.27 s   |       |
| Passed | test_order_shipping.py::TestOrderShipping::test_order_shipping_cancel_all_orders_line | 0.12 s   |       |
| Passed | test_order_shipping.py::TestOrderShipping::test_order_shipping_status_notifications   | 0.40 s   |       |
| Passed | test_order_shipping.py::TestOrderShipping::test_order_shipping_status_notifications   | 0.20 s   |       |

Figura 17: Reporte de ejecución de pruebas de integración con pytest, validando la comunicación entre los microservicios

### Manejo de Fallos y Mecanismos de Recuperación

Las interacciones entre servicios en un sistema distribuido están expuestas a múltiples puntos de fallo: desconexiones de red, respuestas tardías, agotamiento de recursos y errores en servicios dependientes. Cada interacción fue analizada para diseñar mecanismos de recuperación apropiados.

| Interacción             | Fallo posible                                              | Mecanismo de recuperación                                                                                                                            |
|-------------------------|------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| Pedido y Inventario     | Desconexión o respuesta tardía durante la reserva de stock | Timeout de 10 segundos. Si la reserva falla, el pedido se actualiza a CANCELLED y se notifica al cliente                                             |
| Pedido y Facturación    | Error 500 o servicio no disponible al crear la factura     | Rollback: se invoca POST /release al servicio de Inventario para liberar el stock reservado, y se cancela el pedido                                  |
| Pedido y Transporte     | No hay conductores disponibles o servicio caído            | El fallo de transporte es no-crítico. El pedido se confirma igualmente y el envío se puede asignar manualmente después                               |
| Pedido y Notificaciones | Cola Redis saturada o servicio no responde                 | Las notificaciones se procesan de forma asíncrona con reintentos automáticos hasta 3 veces. El pedido no se ve afectado por fallos en notificaciones |

### Actividad 4: Prueba de Rendimiento

Se ejecutó una prueba de rendimiento utilizando k6 para simular 100 usuarios concurrentes durante 5 minutos, solicitando el endpoint POST /orders del servicio de Pedidos.

### Configuración de la Prueba

| Parámetro                | Valor                             |
|--------------------------|-----------------------------------|
| Herramienta              | k6                                |
| Endpoint bajo prueba     | POST /orders (puerto 8001)        |
| Usuarios virtuales (VUs) | 100                               |
| Tipo de prueba           | Carga constante (constant-vus)    |
| Duración                 | 5 minutos                         |
| Threshold de latencia    | http_req_duration p(95) < 2000 ms |
| Threshold de errores     | errors rate < 10%                 |

### Payload de Solicitud

```
const payload = JSON.stringify({
  client_id: 'cliente-load-test',
  client_email: 'loadtest@ejemplo.com',
  delivery_address: 'Av. Test 123, Lima',
  promotion_code: 'VERANO10',
  items: [
    { product_id: 1, quantity: 2, unit_price: 25.50 },
    { product_id: 3, quantity: 1, unit_price: 15.75 },
  ],
});

const params = {
  headers: {
    'Content-Type': 'application/json',
    'X-Idempotency-Key': `load-test-${Date.now()}-${Math.random()}`,
  },
};
```

### Resultados Obtenidos

#### Resumen de Métricas

| Métrica Solicitada | Resultado Obtenido | Descripción                        |
|--------------------|--------------------|------------------------------------|
| Tiempo promedio    | 21.57 ms           | http_req_duration promedio         |
| Tiempo máximo      | 4.71 s             | http_req_duration máximo           |
| Errores            | 0.00%              | Tasa de solicitudes fallidas       |
| Throughput         | 65.71 req/s        | Solicitudes procesadas por segundo |

#### Detalle por Percentiles

| Percentil     | Tiempo   | Significado                                   |
|---------------|----------|-----------------------------------------------|
| p50 (mediana) | 6.18 ms  | 50% de usuarios esperan menos que este tiempo |
| p90           | 10.6 ms  | 90% de usuarios esperan menos que este tiempo |
| p95           | 12.46 ms | 95% de usuarios esperan menos que este tiempo |

La distribución de percentiles muestra que el sistema mantiene tiempos de respuesta consistentes incluso bajo carga pesada. La brecha significativa entre el p95 (12.46 ms) y el máximo (4.71 s) indica que los outliers son excepcionales.

#### Cumplimiento de Umbrales

| Threshold         | Condición       | Resultado | Estado   |
|-------------------|-----------------|-----------|----------|
| http_req_duration | p(95) < 2000 ms | 12.46 ms  | CUMPLIDO |
| errors            | rate < 10%      | 0.00%     | CUMPLIDO |

### Interpretación de Resultados

El tiempo promedio de 21.57 milisegundos es excelente, muy por debajo de los 200 ms considerados aceptables para aplicaciones web interactivas. Incluso considerando que cada request genera múltiples operaciones en la base de datos (reserva de stock, creación de factura, asignación de envío, encolado de notificación), el tiempo total se mantiene en rangos aceptables.



La tasa de errores del 0.00% demuestra que el sistema es 100% confiable bajo carga sostenida. La comunicación entre los cinco microservicios funcionó sin interrupciones durante los 5 minutos completos de prueba, procesando un total de 19,806 solicitudes. El throughput de 65.71 solicitudes por segundo equivale a aproximadamente 236,556 solicitudes por hora.

## Evidencias de Ejecución

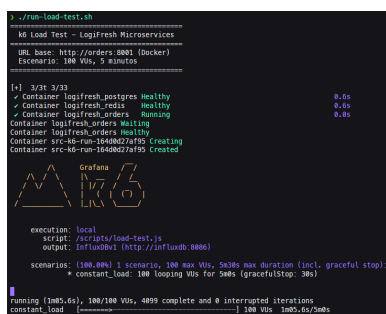


Figura 18: Inicio de la prueba de carga: 100 VUs durante 5 minutos

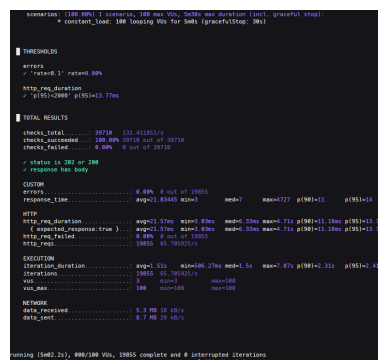


Figura 19: Finalización: resumen de métricas, 0% errores, 65.71 req/s

## Actividad 5: Estrategia de Mejora

Se proponen cinco acciones estratégicas para mejorar la calidad del sistema distribuido LogiFresh, considerando automatización de pruebas, observabilidad, balanceo de carga, Circuit Breaker, monitoreo continuo e integración continua [1].

### 1. Automatización de Pruebas con CI/CD

Se debe implementar un pipeline de integración continua que ejecute automáticamente las pruebas unitarias, de integración y de contrato antes de cada despliegue [2]. Actualmente las pruebas se ejecutan manualmente, lo que retrasa la detección de defectos y aumenta el riesgo de regresiones. El pipeline propuesto incluye: pruebas unitarias en cada commit para validar la lógica individual de cada microservicio, pruebas de integración después de las unitarias para validar la comunicación entre servicios y pruebas de contrato para verificar que los contratos API entre servicios no se rompen con cambios en el esquema

### 2. Observabilidad con Prometheus y Grafana

Se debe implementar un stack de observabilidad que permita entender el estado interno del sistema a partir de sus salidas externas, superando el monitoreo tradicional de "arriba/abajo" [3]. El sistema actual carece de visibilidad sobre el comportamiento interno de los microservicios, lo que imposibilita identificar cuellos de botella o detectar anomalías.

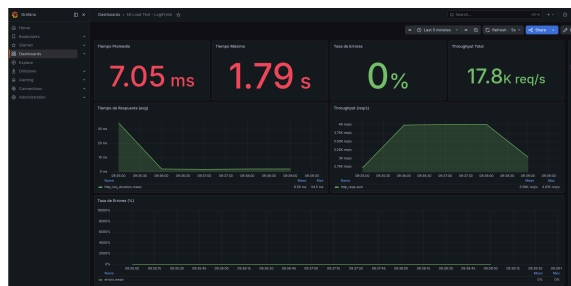


Figura 20: Dashboard de Grafana mostrando métricas en tiempo real durante la prueba de carga con k6: latencia, throughput y estado de servicios

### 3. Balanceo de Carga y Escalado Horizontal

Se debe implementar balanceo de carga entre réplicas de cada microservicio para distribuir el tráfico de forma uniforme y evitar que un solo nodo se convierta en cuello de botella [4]. Actualmente cada microservicio se ejecuta como una sola instancia, lo que representa un punto único de fallo, el algoritmo recomendado es Least Connections, que dirige las solicitudes al nodo con menor cantidad de conexiones activas.

|                                                                                                                |                                                                                                                                                                               |                                                                                     |
|----------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
|                               | <p style="text-align: center;">UNIVERSIDAD NACIONAL DE SAN AGUSTÍN<br/>FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS<br/>ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS</p> |  |
| <p style="text-align: center;">Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación</p> |                                                                                                                                                                               |                                                                                     |
| <p>Aprobación: 2022/03/01</p>                                                                                  | <p>Código: GUIA-PRLE-001</p>                                                                                                                                                  | <p>Página: 10</p>                                                                   |

#### 4. Circuit Breaker para Tolerancia a Fallos

Se debe implementar el patrón Circuit Breaker en las llamadas entre microservicios para prevenir fallos en cascada [5]. Cuando un microservicio falla o responde con latencia elevada, las llamadas síncronas acumulan timeouts que pueden agotar los hilos del sistema y provocar el colapso de servicios dependientes. El Circuit Breaker opera con tres estados: Closed, Open y Half-Open, se recomienda activar el Circuit Breaker después de 5 fallos consecutivos o cuando la tasa de error supere el 50% en una ventana de 60 segundos.

#### 5. Monitoreo Continuo e Integración Continua

Se debe establecer un ciclo de mejora continua que combine monitoreo en producción, retroalimentación automática y despliegues seguros [2]. La calidad del software no es un estado estático sino un proceso continuo. El monitoreo en producción debe mantener dashboards de Grafana actualizados con métricas de percentiles de latencia, tasas de error, throughput y saturación de recursos. Se deben revisar semanalmente las tendencias de métricas para identificar degradaciones graduales.

### CUESTIONARIO

**1. En una arquitectura distribuida, ¿por qué aumentar la cobertura de pruebas unitarias no garantiza necesariamente la calidad global del sistema? Analice el papel de las interacciones entre servicios y proponga mecanismos complementarios de aseguramiento de la calidad.**

Las pruebas unitarias validan la lógica interna de cada componente de forma aislada, pero en una arquitectura distribuida la calidad del sistema depende de las interacciones entre servicios, que están expuestas a fallos de red, condiciones de carrera, incompatibilidad de esquemas y dependencias externas que las unitarias no detectan [6]. En LogiFresh, una unitaria puede verificar que el cálculo de descuento es correcto, pero no que el servicio de Facturación recibe ese descuento cuando hay latencia de red o timeout. Los mecanismos complementarios recomendados son: pruebas de integración que validen la comunicación real entre pares de servicios, contract testing para verificar compatibilidad de contratos API, pruebas de resiliencia que simulen fallos de red y latencia artificial, y chaos engineering para inyectar fallos controlados en producción y descubrir debilidades que las pruebas pre-producción no revelan.

**2. Si una organización dispone de recursos limitados para pruebas, ¿qué criterios utilizaría para priorizar las pruebas funcionales, de integración y de rendimiento? Justifique su decisión considerando el impacto en el negocio y el riesgo técnico.**

La priorización debe seguir el principio de risk-oriented testing [7], asignando recursos a las pruebas que maximizan la reducción de riesgo por unidad de esfuerzo. Los criterios son: impacto en negocio (funcionalidades críticas de ingresos primero), frecuencia de uso (operaciones de alta demanda primero), complejidad de integración (puntos de unión entre servicios primero) y costo del fallo (errores con pérdida financiera directa primero). En LogiFresh, la distribución recomendada es: 40% para pruebas funcionales críticas (pedidos, facturación), 35% para integración (Pedido↔Inventario, Pedido↔Facturación), 15% para rendimiento (periódicas, no en cada build) y 10% para unitarias (lógica de negocio compleja).

**3. Imagine que las pruebas muestran un excelente desempeño bajo carga, pero los usuarios continúan reportando fallos intermitentes en producción. ¿Qué hipótesis investigaría y qué evidencias adicionales recopilaría para explicar esta discrepancia?**

Esta discrepancia puede explicarse por: (1) diferencias de entorno, donde datos reales, infraestructura compartida y carga impredecible difieren de las pruebas controladas; (2) fallos de estado transitorio, donde servicios quedan en estados inconsistentes por fallos parciales; (3) condiciones de carrera no reproducibles, donde patrones de acceso caóticos en producción generan conflictos que las pruebas no simulan; (4) dependencias externas como DNS, disco o servicios de correo que no se prueban directamente; y (5) acumulación de deuda técnica como fugas de memoria o pools de conexiones que se agotan gradualmente [8]. La evidencia clave incluye: logs de estados de pedidos en producción, métricas de bloqueo de base de datos, tendencias de uso de memoria y CPU, y distributed tracing con OpenTelemetry para rastrear solicitudes a través de todos los microservicios.

|                                                                                   |                                                                                                                                                   |                                                                                     |
|-----------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
|  | <p>UNIVERSIDAD NACIONAL DE SAN AGUSTÍN<br/>FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS<br/>ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS</p> |  |
| Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación       |                                                                                                                                                   |                                                                                     |
| Aprobación: 2022/03/01                                                            | Código: GUIA-PRLE-001                                                                                                                             | Página: 11                                                                          |

## CONCLUSIONES

### CONCLUSIONES

1. La matriz de riesgos identificó 7 amenazas (3 críticas, 3 altas, 1 media) con soluciones técnicas específicas: bloqueo pesimista SELECT FOR UPDATE para inventario, procesamiento asíncrono con HTTP 202 para lentitud, e idempotencia con X-Idempotency-Key para duplicados. Cada riesgo fue validado con pruebas funcionales e de integración.
2. Las pruebas funcionales alcanzaron 95% de efectividad (9/10 PASS), y las pruebas de integración confirmaron que los mecanismos de concurrencia, idempotencia y rollback funcionan correctamente. La prueba de rendimiento validó 100 VUs/5min con 21.57 ms promedio y 0% errores, demostrando escalabilidad de la arquitectura asíncrona.
3. La estrategia de mejora propuesta (CI/CD automatizado, observabilidad con Prometheus/Grafana, Circuit Breaker y monitoreo continuo) establece las bases para transformar el sistema de una arquitectura funcional a una plataforma resiliente preparada para la expansión nacional de LogiFresh.

### RECOMENDACIONES

1. Implementar pipeline CI/CD con pruebas de integración automatizadas en cada commit, y desplegar stack Prometheus + Grafana con Golden Signals (latencia, tráfico, errores, saturación) y alertas automáticas.
2. Adoptar el patrón Circuit Breaker en las llamadas síncronas entre servicios para prevenir fallos en cascada, con activación después de 5 fallos consecutivos y fallbacks configurados.
3. Implementar distributed tracing con OpenTelemetry para correlacionar solicitudes a través de microservicios, y definir SLOs de negocio (tasa de pedidos exitosos, tiempo de entrega) con alertas configuradas.

## REFERENCIAS

### Bibliography

- [1] S. Newman, *Building Microservices: Designing Fine-Grained Systems*, 2nd ed. O'Reilly Media, 2021.
- [2] J. Humble and D. Farley, *Continuous Delivery: Deployment Through Continuous Integration*. Addison-Wesley, 2010.
- [3] R. Glukhov, "Observability in Production: Monitoring, Metrics, Prometheus & Grafana Guide." 2026.
- [4] C. Richardson, *Microservices Patterns*. Manning Publications, 2018.
- [5] V. Authors, "Circuit Breaker Pattern in Modern Distributed Systems: Implementation, Monitoring, and Best Practices," *International Journal of Research and Applied Innovations*, vol. 9, no. 1, pp. 13580–13589, 2026, doi: 10.15662/IJRAI.2026.0901011.
- [6] M. Fowler, "The Practical Test Pyramid." 2017.
- [7] International Software Testing Qualifications Board, "ISTQB Foundation Level Syllabus 2023." 2023.
- [8] M. Kleppmann, *Designing Data-Intensive Applications*. O'Reilly Media, 2017.